

PYTHON STANDARD LIBRARY

MODULE CSV

Introduction

The **CSV module** is a powerful built-in tool designed to parse and generate files in Comma Separated Values format.

- ✓ Native Implementation: Included in the Python standard library, requiring no installation.
- ✓ Versatile Dialects: Supports multiple formatting styles (Excel, custom delimiters, etc.).
- ✓ Data Integrity: Handles special characters and quoting automatically.



Purpose of the Module



Standardization

Automates the parsing of complex formats, including Excel dialects, custom delimiters, and multi-line fields



Efficiency

Uses iterators to process files line-by-line, ensuring minimal memory footprint even with multi-gigabyte datasets



Accessibility

A built-in module with zero external dependencies, making it the most portable choice for cross-platform scripts

Functions & Classes

csv.reader(file)

Reads a file and gives you the data row by row. Each row comes out as a simple Python list (like ['Name', 'Age', 'City']).

csv.writer(file)

Creates a tool that lets you write your Python lists into a file, automatically adding the commas between the items.

csv.register_dialect(name)

Lets you save a custom set of rules (like "use a semicolon instead of a comma") and give it a nickname.

csv.unregister_dialect(name)

Deletes a custom set of rules you saved earlier.

csv.get_dialect(name)

Grabs the custom set of rules you saved so you can look at them.

csv.list_dialects()

Shows you a list of all the nicknames for the custom rules currently available.

csv.field_size_limit()

Tells you (or lets you change) the maximum amount of text allowed in a single CSV box (cell). This is helpful if you have a file with massive paragraphs that is causing Python to crash.

csv.Error

The specific error message Python will throw if something goes completely wrong while trying to read or write your CSV file.

Functions & Classes

`csv.DictReader(file)`

Reads a file and turns every row into a Python dictionary. It automatically uses the very first row of your file as the keys (headers).

`csv.DictWriter(file, fieldnames)`

Takes your Python dictionaries and writes them into a file. You have to tell it the 'fieldnames' (headers) first so it knows what order to put the columns in.

`csv.Sniffer`

It acts like a detective. If someone gives you a weird CSV file and you don't know if they used commas, tabs, or semicolons, the Sniffer will read a small sample and guess the rules for you. It can also guess if the first row is a header.

`csv.Dialect`

The master blueprint used to build custom formatting rules.

`csv.excel`

The standard format used by Microsoft Excel (this is the default rule Python uses if you don't say otherwise).

`csv.excel_tab`

The format Excel uses when items are separated by hitting the "Tab" key instead of a comma.

`csv.unix_dialect`

A specific format used mostly by Mac and Linux systems for how they end a line of text.

Syntax and Examples

Reading Data

```
import csv

with open('data.csv', 'r') as f:
    reader = csv.reader(f)
    for row in reader:
        print(row) # Output is a list
```

Writing Data

```
import csv

with open('out.csv', 'w', newline='') as f:
    wr = csv.writer(f)
    wr.writerow(['ID', 'Role'])
    wr.writerows([[1, 'Admin'], [2, 'Dev']])
```

Real-world Applications



Log Analysis: Parsing server and application logs for troubleshooting and security auditing.



ETL Processes: Extracting database records for migration or data warehouse ingestion.



Financial Reporting: Generating statements and transaction summaries from banking systems.



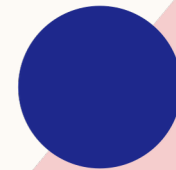
E-commerce: Bulk updating product prices, descriptions, and stock levels across platforms.



CALENDAR PRESENTATION

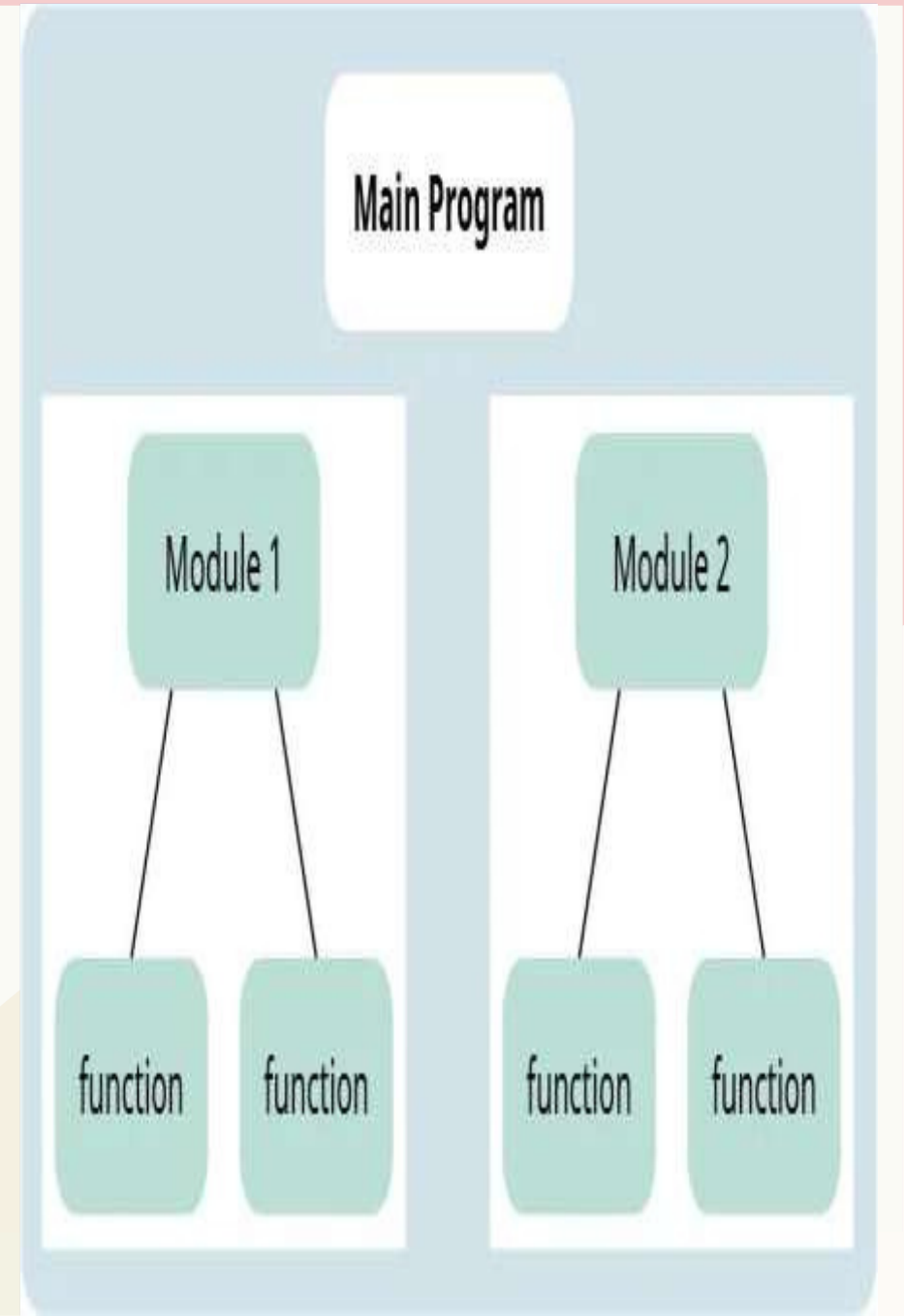
AGENDA

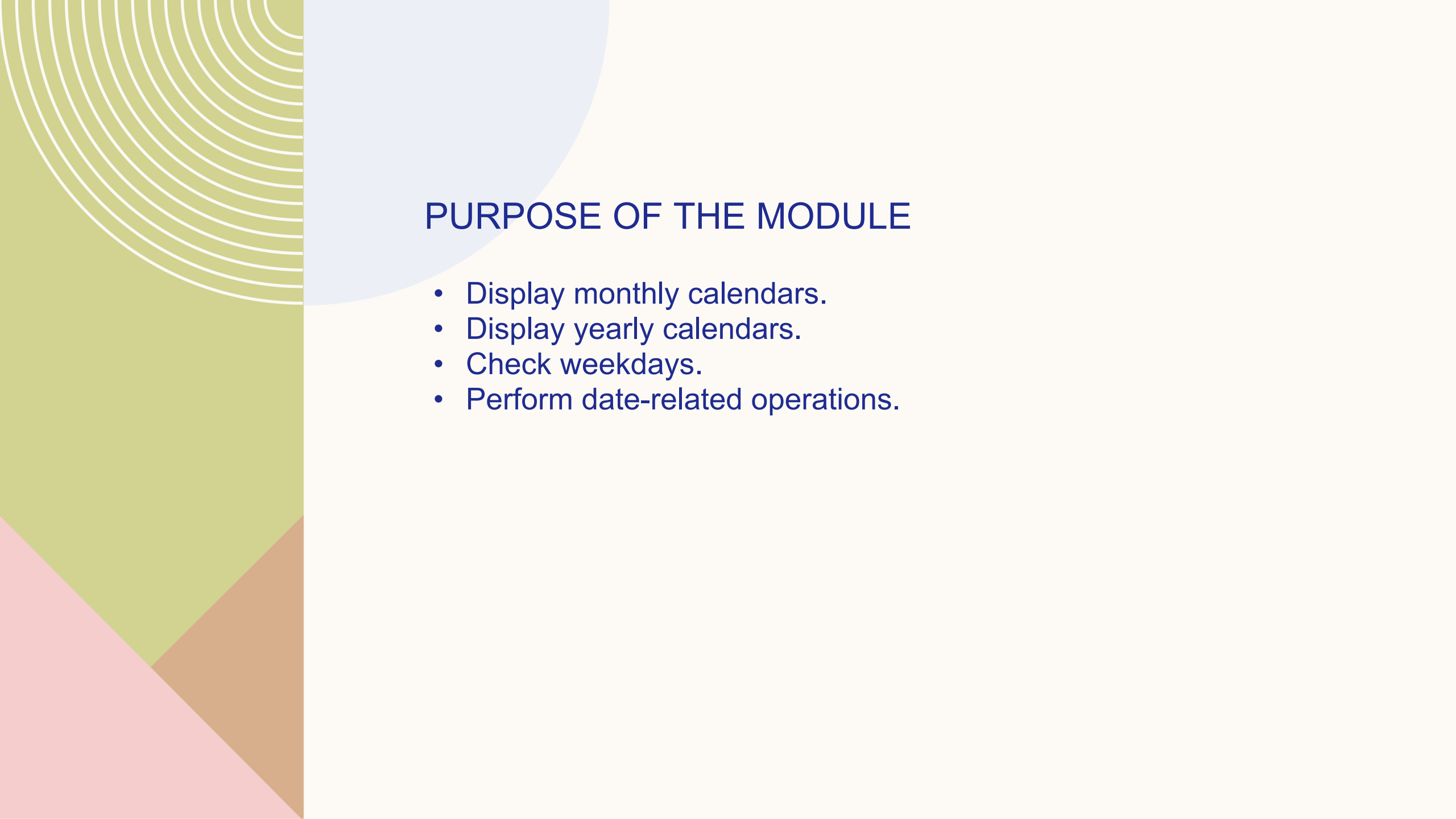
1. Module Introduction
2. Purpose of the Module
3. Important Functions/Classes
4. Syntax and Examples
5. Real-world Applications



INTRODUCTION

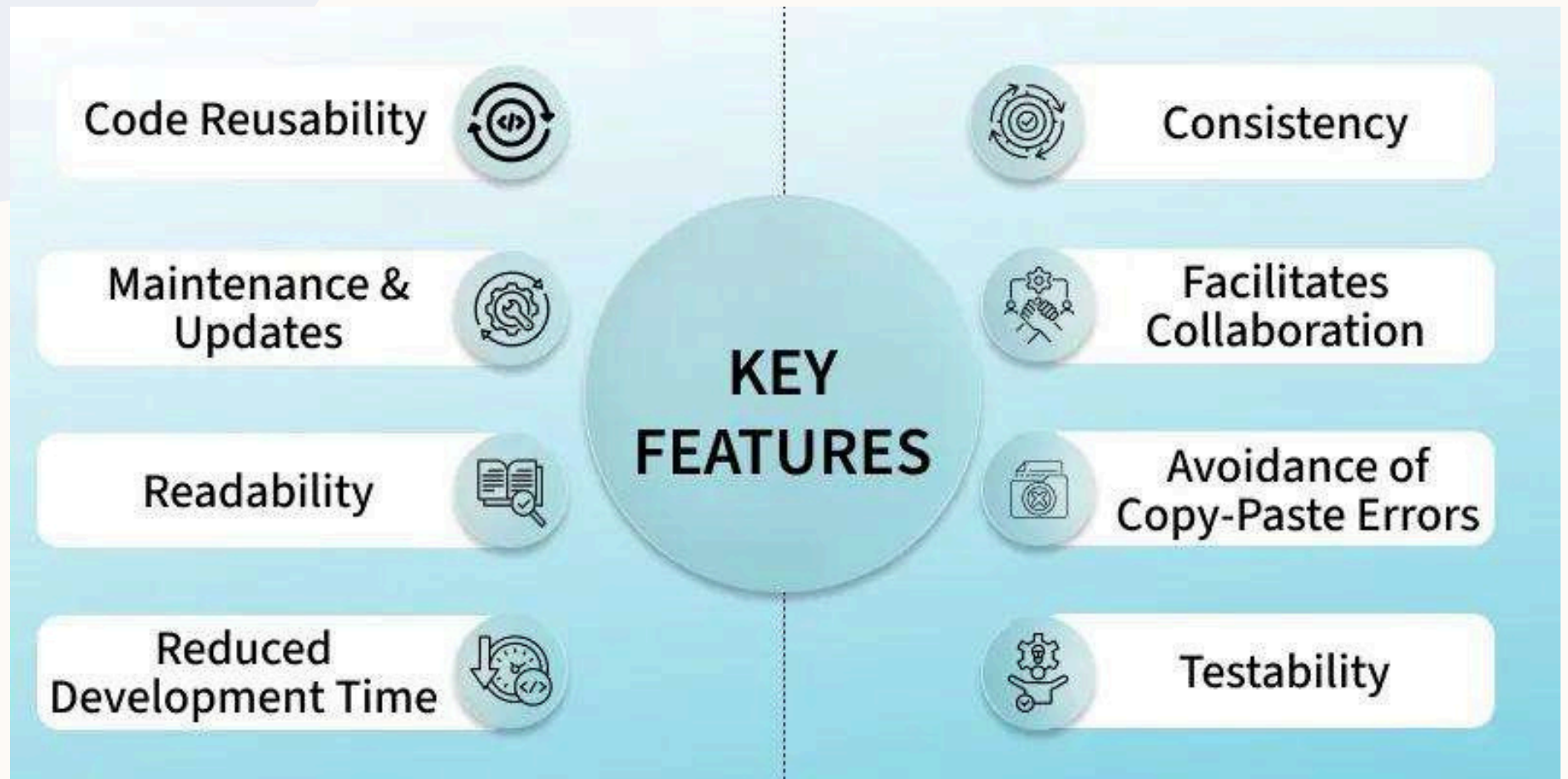
- 1.Calendar is a built-in Python module.
- 2.It helps work with dates, months, and years.
- 3.It can display calendars in different formats.

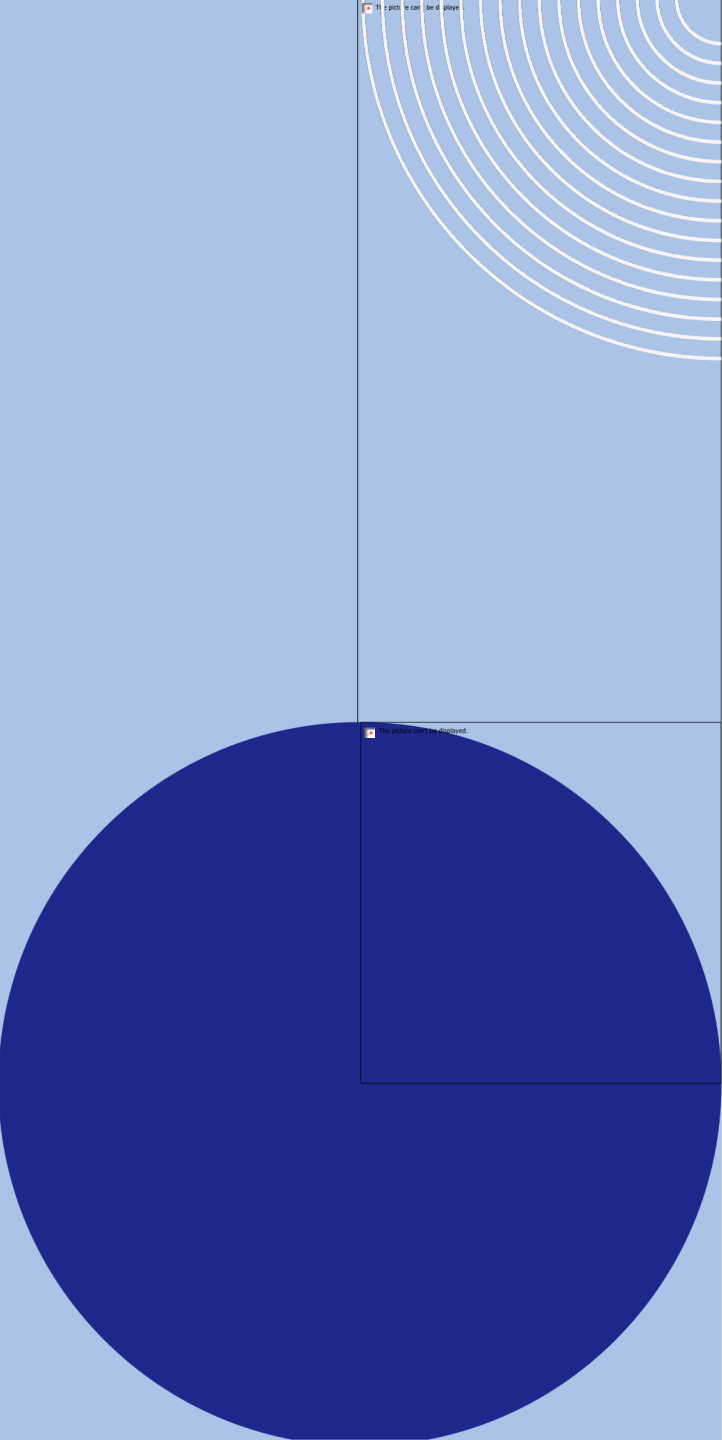




PURPOSE OF THE MODULE

- Display monthly calendars.
- Display yearly calendars.
- Check weekdays.
- Perform date-related operations.





IMPORTANT FUNCTIONS/CLASSES

calendar.month()

calendar.calendar()

calendar.weekday()

calendar.isleap()

SYNTAX AND EXAMPLES

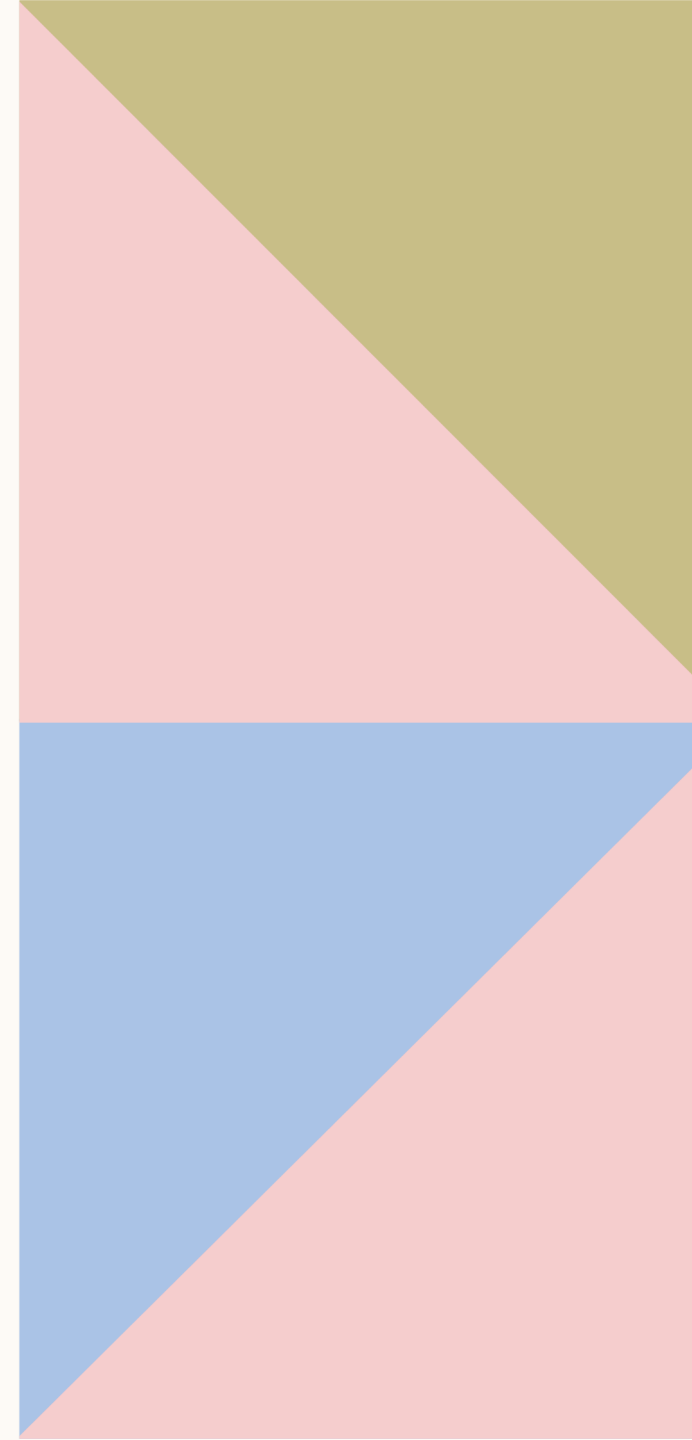
```
import calendarprint(calendar.month(2026, 6))
```

OUTPUT:

June 2026

Mo Tu We Th Fr Sa Su

1 2 3 4 5 6 7...



REAL-WORLD APPLICATIONS

- Attendance systems
- Event management
- Booking systems
- Scheduling applications
- Reminder applications



THANK YOU!!!

1. Orike Richa
2. Somya Ranjan Sahoo